Repositorio de TDAs
(usando CLI / VSCode / Eclipse)

Estructuras de Datos - DCIC - UNS

Introducción

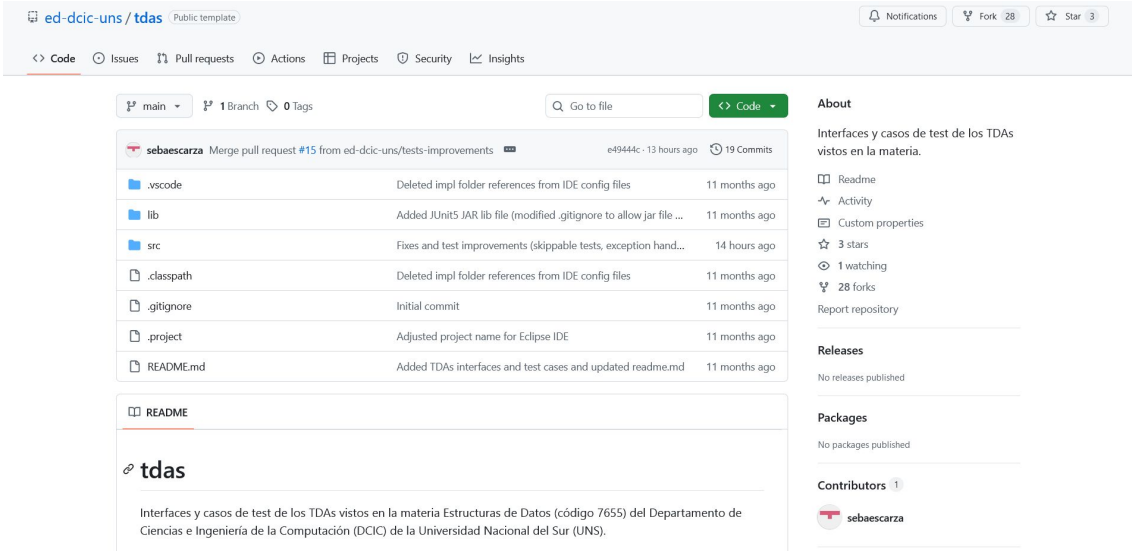
Los archivos de la materia se encuentran disponibles en un repositorio público Git en GitHub.

Dicho repositorio:

- Contiene las interfaces de los TDAs a implementar durante el cuatrimestre
- Contiene un conjunto de casos de prueba para testear las implementaciones de los TDAs a realizar como parte de los trabajos prácticos de la materia
- Constituye proyectos pre-configurados para ser utilizados en Visual Studio y/o Eclipse.
- Trae JUnit pre-configurado y listo para usar.

Obteniendo los archivos del repositorio

El repositorio Git en Github de la materia se encuentra en la siguiente dirección: https://github.com/ed-dcic-uns/tdas/



Obteniendo los archivos del repositorio

Hay tres formas de obtener los archivos de un repositorio en GitHub:

- Sin cuenta en GitHub, la única forma es **descargando** la última versión de una rama **como un archivo .ZIP.** Es el método más simple pero más limitado ya que no generamos un repositorio para trabajar el código y no usamos git.
- Con cuenta en GitHub, **haciendo un fork** del repositorio de la materia, y luego clonando ese repositorio localmente. Los repositorios quedan vinculados al repositorio de la materia para actualizaciones y aportes.
- **[RECOMENDADA]** Con cuenta en GitHub, **creando un nuevo repositorio remoto usando** el de la materia como **template**, y luego clonando ese repositorio localmente. Esto crea repositorios independientes del repositorio de la materia pero con su misma estructura y archivos.

Descarga .zip

Obteniendo los archivos - Descarga .zip

Es la forma más simple de obtener los archivos.

Dado que el repositorio es público, no requiere la creación de una cuenta en GitHub.

Permite replicar la estructura de carpetas y archivos de la versión de una rama del repositorio, pero no genera un repositorio local y no permite utilizar Git/Github para el control de versiones.

Útil como primera aproximación para obtener rápidamente archivos puntuales (cambios acotados), o la totalidad de los archivos pero como paso previo a crear y configurar una cuenta en Github.

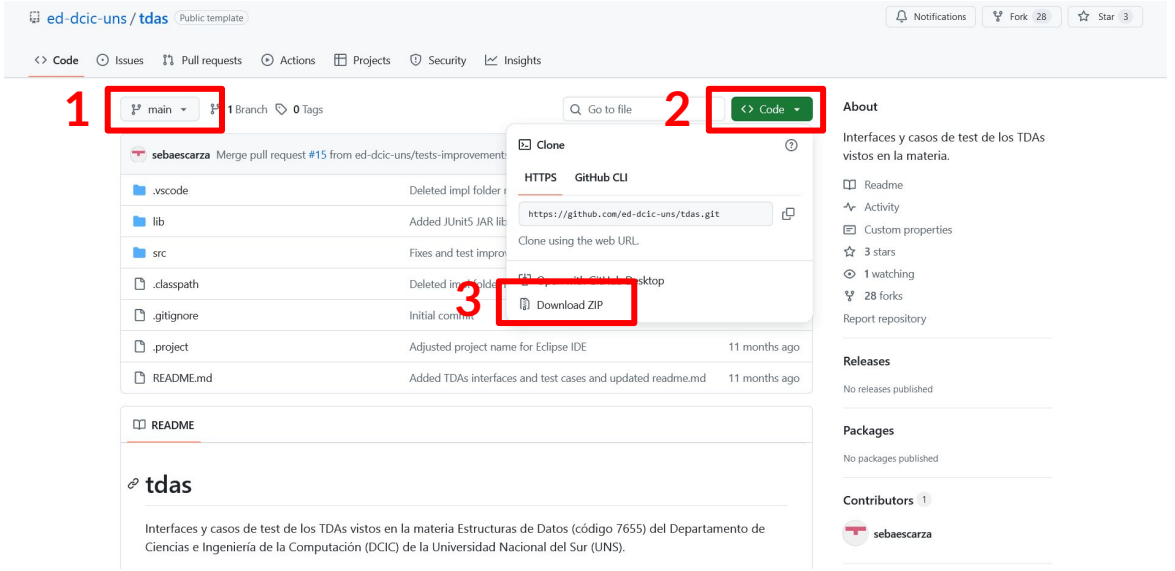
No se recomienda como método de obtención de archivos definitivo.

Obteniendo los archivos - Descarga .zip

1. Confirmar que la rama elegida es **main**
2. Desplegar el menú del botón **<> Code**.
3. Seleccionar la opción **Download ZIP**

Extraer el archivo descargado para obtener el contenido.

La carpeta **tdas-main** es la carpeta a abrir como base o proyecto con el IDE.

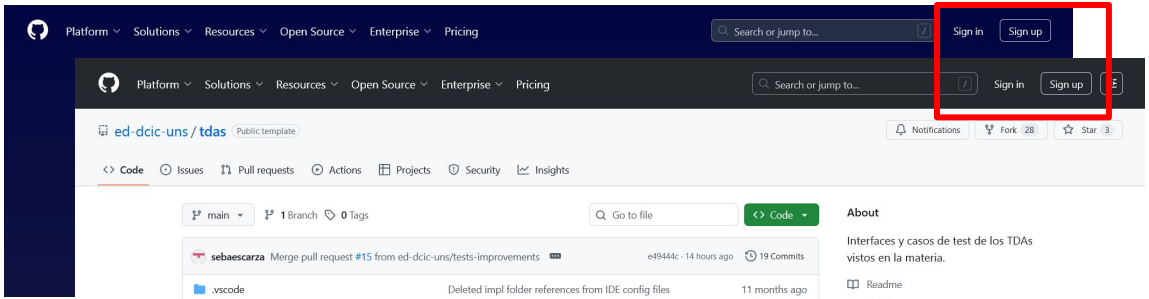


Creando una cuenta en GitHub

Creando/configurando una cuenta en GitHub

Las otras alternativas de obtención de archivos del repositorio involucran la creación de un repositorio propio en GitHub, por lo que requieren la creación de una cuenta y su configuración en la plataforma.

1. Dirigirse a https://github.com/ o bien a https://github.com/ed-dcic-uns/tdas/
2. En la parte superior derecha se encuentra la opción **Sign Up** para registrarse si no se tiene cuenta, o **Sign In** si ya se posee una.
3. Seguir las instrucciones en pantalla para crear la cuenta o ingresar si ya se posee una.

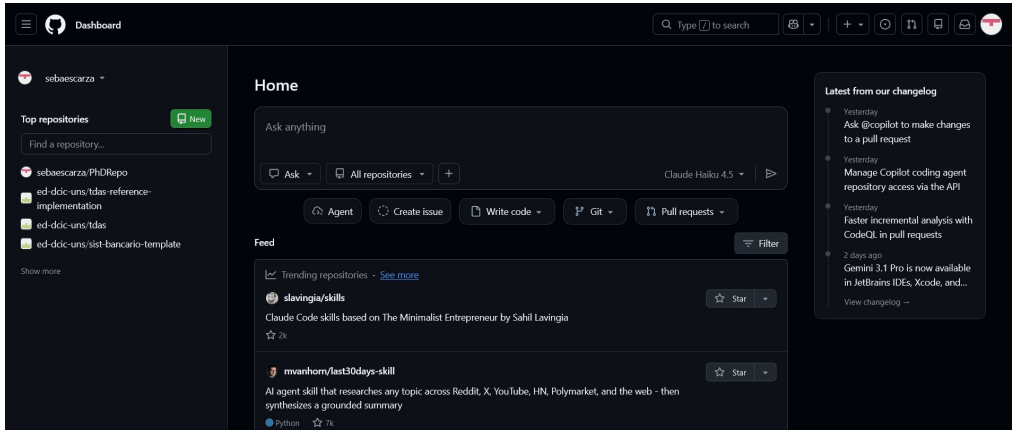


Creando/configurando una cuenta en GitHub

GitHub centraliza toda la actividad como desarrolladores en torno a repositorios, proyectos, issues (tareas), pull requests, etc. Al ingresar se muestra en el dashboard:

- los repositorios propios más activos (izq),
- ventana de chatbot (AI) y un feed con repositorios que son tendencia (centro),
- novedades de github (der).

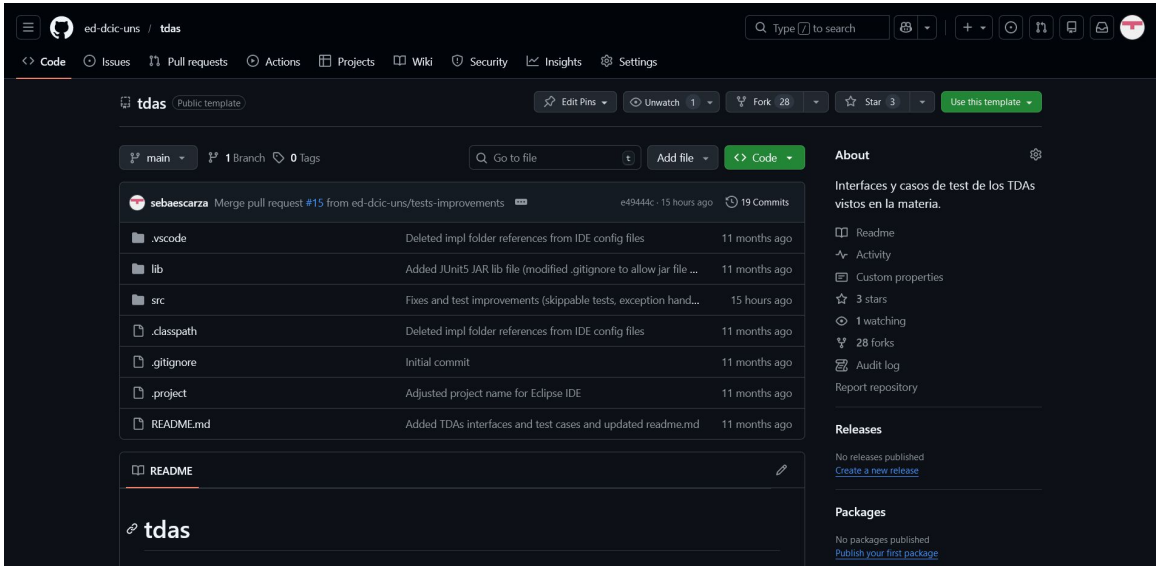
Buscar el repositorio de la materia en el buscador, o pegar la URL en el navegador con la sesión iniciada para acceder.



Creando/configurando una cuenta en GitHub

La vista del repositorio con una sesión iniciada, muestra algunas opciones adicionales que nos permiten obtener los archivos a partir de crear repositorios propios en GitHub:

- Hacer un Fork del repositorio de TDAs
- Usar el repositorio de TDAs como template de uno nuevo



Cómo crear un fork

Obteniendo los archivos - Haciendo un fork

Un fork, o bifurcación, es un nuevo repositorio que es una derivación de un repositorio pre-existente.

Se utiliza mucho en proyectos open source para contribuir y aportar mejoras. Un desarrollador puede hacer un fork de un proyecto abierto, modificar dicho fork introduciendo mejoras, solucionando problemas, y testeando sus cambios, y una vez listo, solicitar que dichos cambios se integren al repositorio original (pull request o PR).

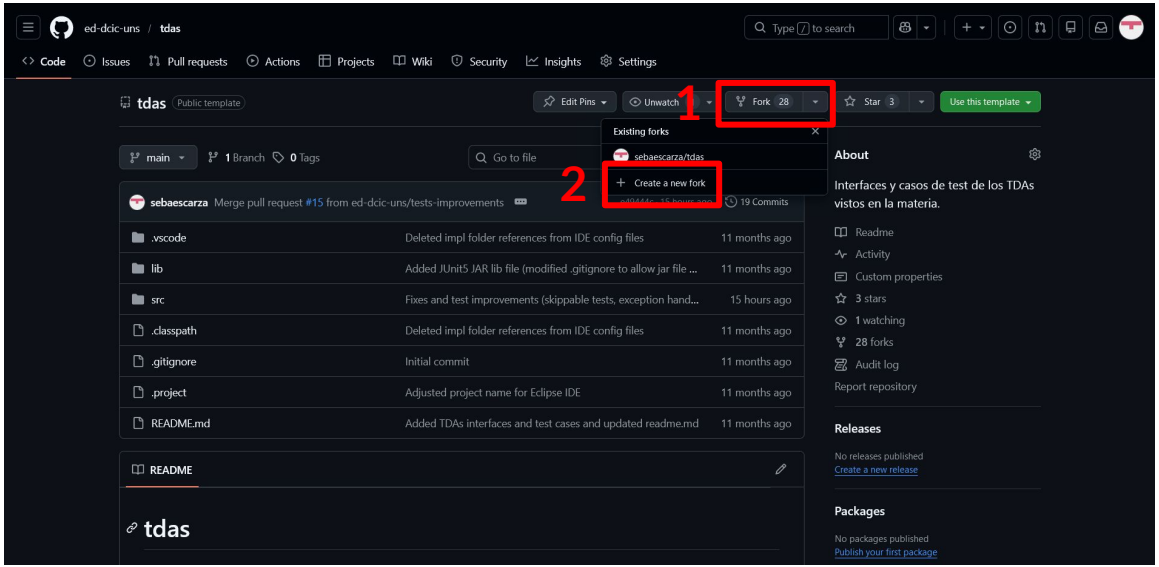
El fork y el repositorio original o base (llamado upstream) quedan conectados: los cambios en el repositorio base pueden actualizarse en el fork, y el fork puede aportar cambios al repositorio base.

No se recomienda hacer un fork del repositorio de TDAs para obtener los archivos ya que no se va a evolucionar dicho repositorio incorporando las implementaciones a realizar en la práctica. Por otra parte, es raro que dicho repositorio se modifique durante el dictado de la materia, por lo que tampoco deberían producirse cambios a incorporar (cambios que en caso de producirse se podrán integrar de otras maneras).

Obteniendo los archivos - Haciendo un fork

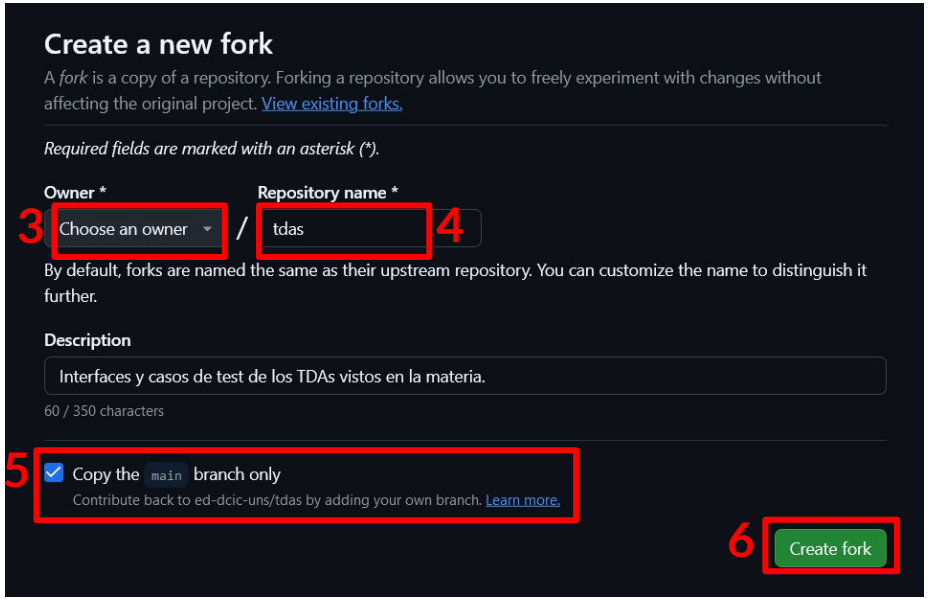
Para crear un fork hay que:

1. Desplegar el menú correspondiente
2. Seleccionar la opción **Create a new fork**



Obteniendo los archivos - Haciendo un fork

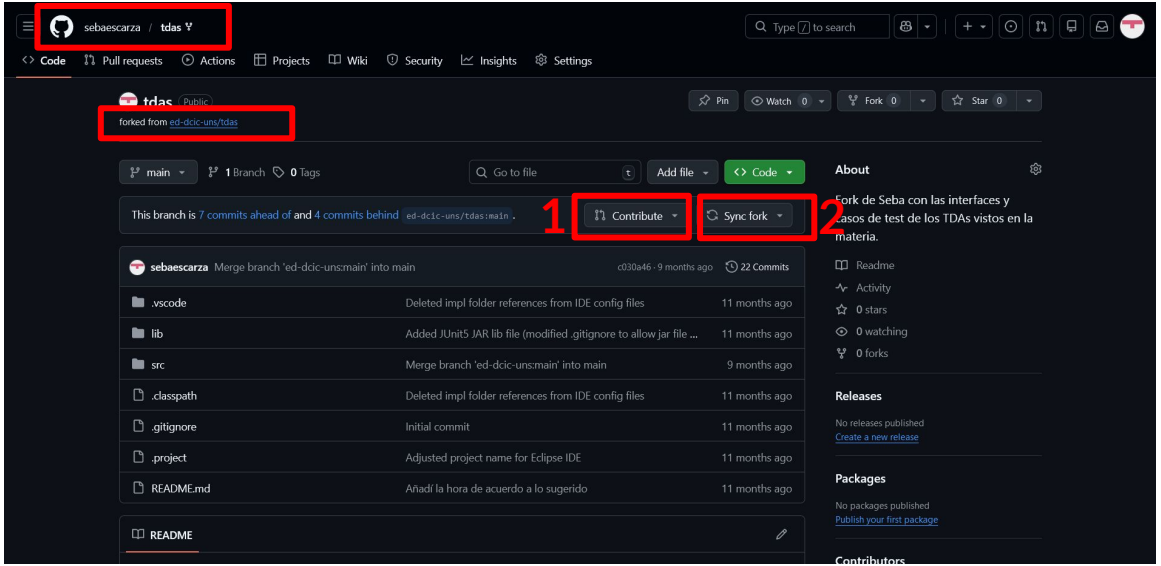
3. Seleccionar un Owner (dueño del fork). Normalmente debería preseleccionarse el usuario creado.
4. Normalmente se deja el mismo nombre para el fork que para el repo original.
5. Se recomienda copiar la rama main solamente. Esto evita copiar otras ramas de desarrollo.
6. Presionar **Create fork**.



Obteniendo los archivos - Haciendo un fork

Como consecuencia de esto se tiene un repositorio propio (no depende de **ed-dcic-uns** sino del usuario) que está conectado al repositorio original:

1. Para contribuir con mejoras
2. Para sincronizar y traer los cambios del repositorio original



Cómo crear un repositorio desde un template (recomendado)

Obteniendo los archivos - Repo desde template

La creación de un repositorio desde un template genera un repositorio que es copia exacta en estructura y archivos del repositorio original. Se requiere que el repositorio original se configure como template para que otros repositorios puedan crearse a partir de éste.

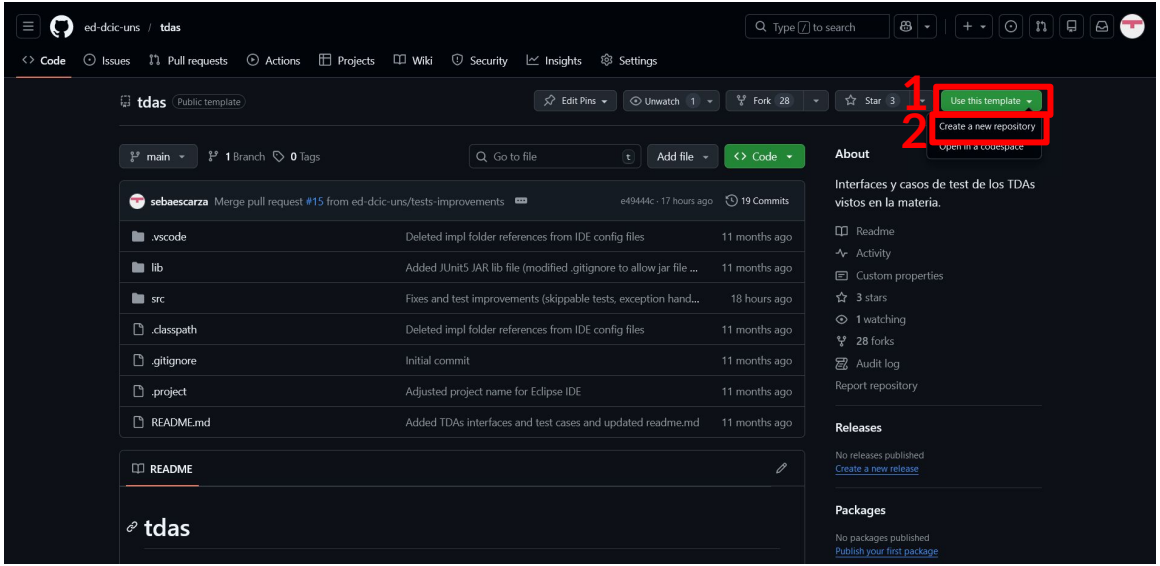
La copia constituye un repositorio equivalente al original y es completamente independiente. Equivale a crear un repositorio desde cero, y luego incorporar la estructura de archivos y carpetas del otro repositorio como si los hubiésemos generado a mano. Una vez creado, ambos repositorios evolucionan de manera independiente.

Es la forma recomendada de utilizar el repositorio de TDAs en la materia.

Obteniendo los archivos - Repo desde template

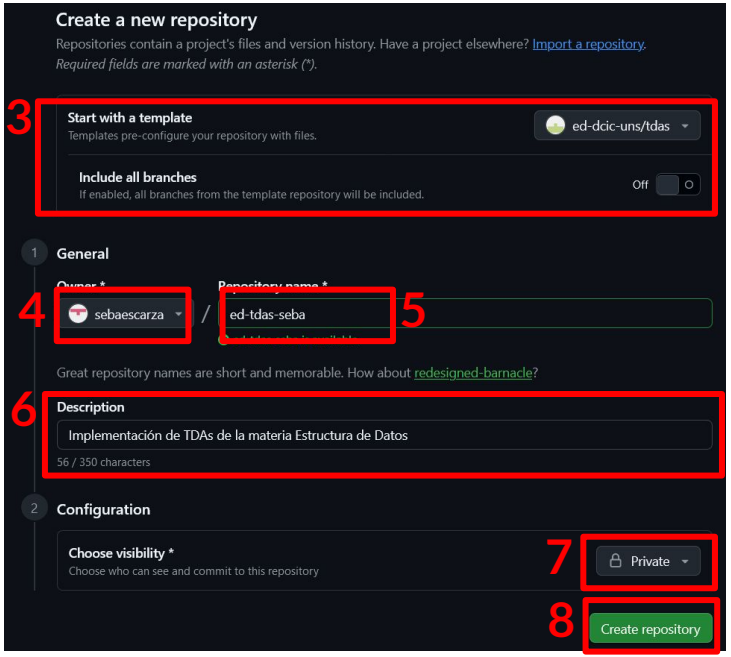
Para crear un repo desde un template hay que:

1. Desplegar el menú correspondiente
2. Seleccionar la opción **Create a new repository**



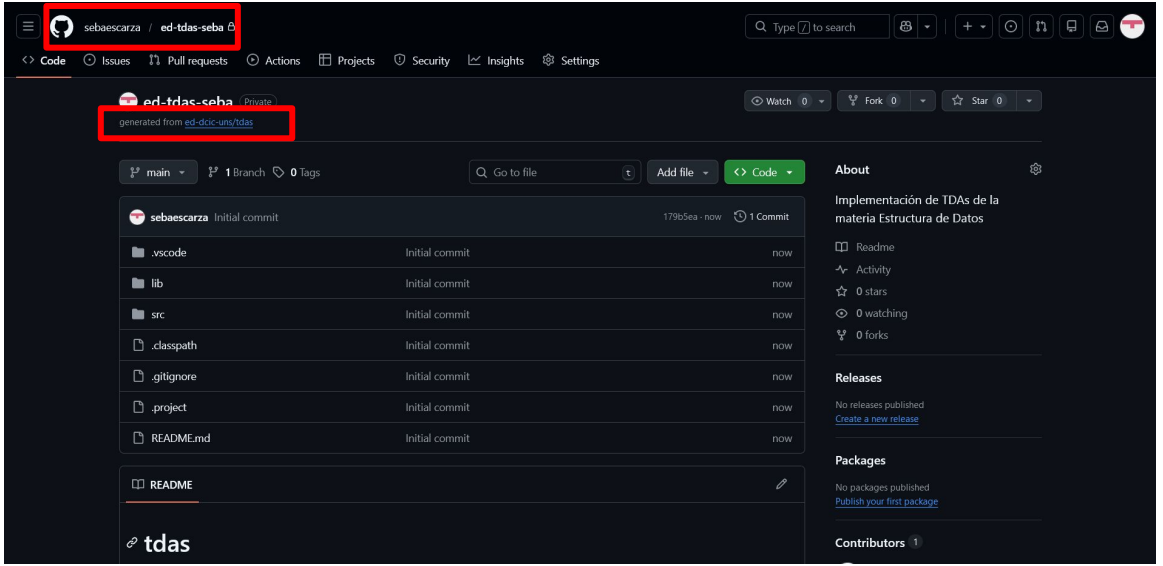
Obteniendo los archivos - Repo desde template

3. La selección del template base se realiza automáticamente. Se recomienda no incluir todas las ramas, evitando copiar otras ramas de desarrollo distintas de main.
4. Seleccionar un Owner (dueño del repo). Debería estar preseleccionado su usuario.
5. Darle un nombre al nuevo repositorio
6. Completar la descripción (opcional)
7. Seleccionar si el repositorio es visible para todo el mundo o no (no posible en forks)
8. Presionar **Create repository**.



Obteniendo los archivos - Repo desde template

Como consecuencia de esto se tiene un repositorio propio (no depende de **ed-dcic-uns** sino del usuario) que es completamente independiente del repositorio original.



Tokens de acceso en GitHub

GitHub - Gestionando el acceso (tokens)

Tanto si se hizo un fork, como si se creó un nuevo repositorio en GitHub a partir de un template, tenemos un repositorio remoto que tenemos que replicar localmente en nuestra computadora para poder trabajarlo. Este paso se logra mediante la clonación.

Pero antes hay que **habilitar el acceso para poder autenticarnos desde nuestra computadora**.

Por razones de seguridad, GitHub usa un sistema de credenciales distinto del usuario y contraseña de la cuenta principal para autenticar los accesos a los distintos repositorios. Esto permite conectar repositorios y herramientas por canales separados que pueden revocarse si ocurren problemas (ej. hackeos o publicación accidental o deliberada de credenciales) sin ver comprometida la cuenta de GitHub.

GitHub - Gestionando el acceso (tokens)

Para esto GitHub maneja tres formas de conectar aplicaciones: GitHub Apps, OAuth Apps y **Tokens de Acceso Personal (PAT)**. Nosotros usaremos esta última opción por simplicidad.

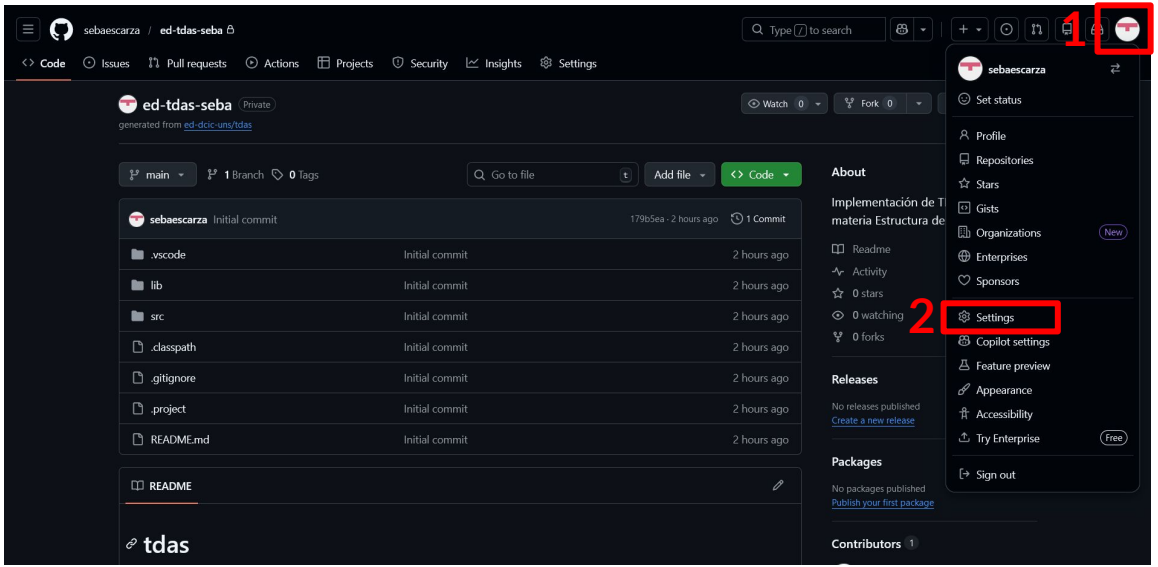
Los PAT pueden ser clásicos o fine-grained. Los clásicos dan acceso básico, mientras que los fine-grained dan más control sobre los permisos que se otorgan. Por simplicidad **utilizaremos PAT Clásicos**.

Cuando intentemos hacer una operación de git contra el repositorio de GitHub (ej. clonar un repositorio privado, o hacer push de commits o ramas en repositorios públicos o privados), el servidor de GitHub nos va a pedir que nos autentiquemos, y ahí **utilizaremos los tokens de acceso que configuremos en lugar del usuario y contraseña de nuestra cuenta de GitHub**.

GitHub - Gestionando el acceso (tokens)

Para configurar un Personal Access Token (PAT) Clásico:

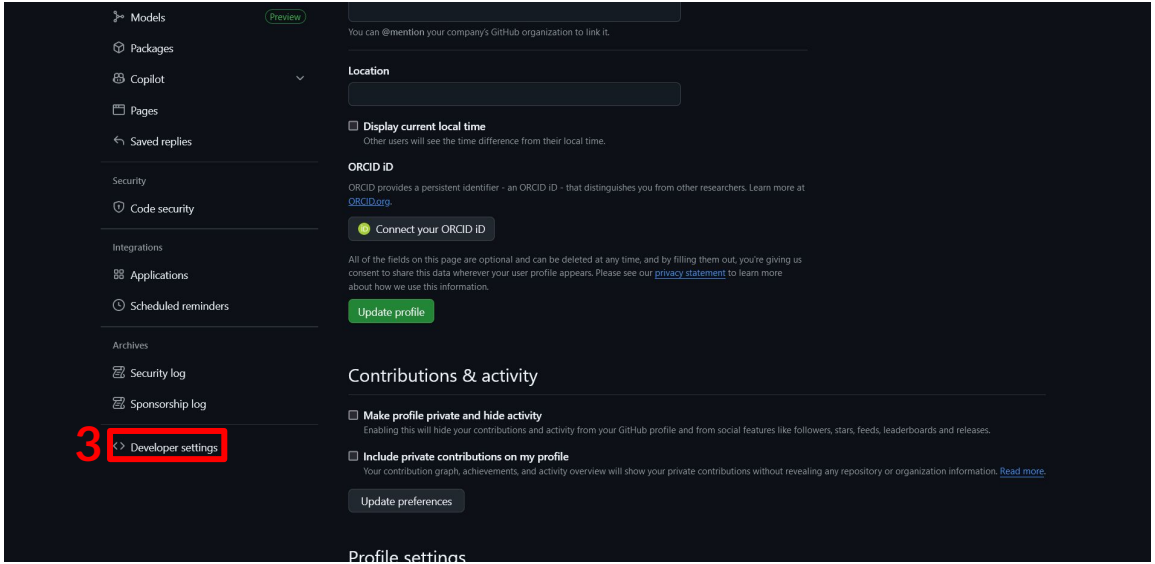
1. Abrir el menú haciendo click en la imagen de la cuenta (esquina superior derecha)
2. Seleccionar **Settings**



GitHub - Gestionando el acceso (tokens)

Para configurar un Personal Access Token (PAT) Clásico:

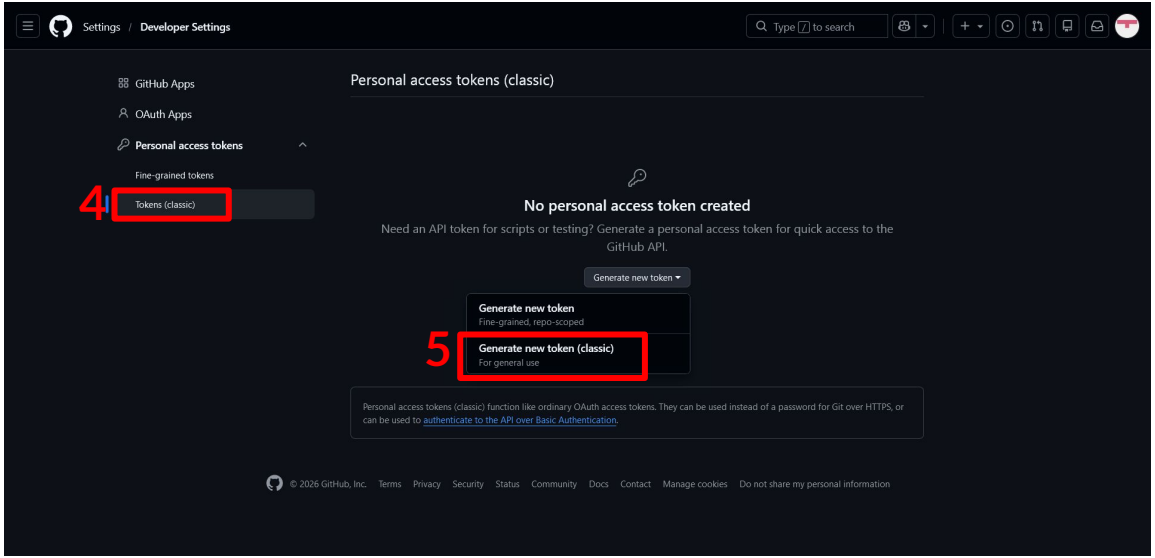
3. Deslizar hasta abajo en el menú de la izquierda y elegir **Developer settings**



GitHub - Gestionando el acceso (tokens)

Para configurar un Personal Access Token (PAT) Clásico:

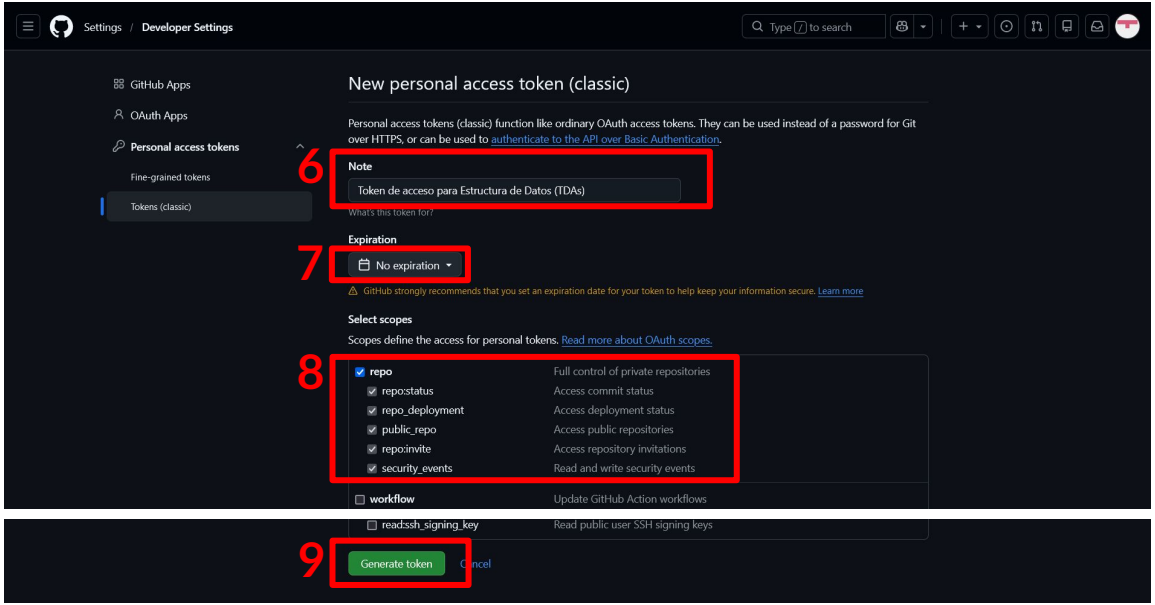
4. Elegir **Token (classic)**
5. Y luego **Generate new token (classic)**



GitHub - Gestionando el acceso (tokens)

Para configurar un Personal Access Token (PAT) Clásico:

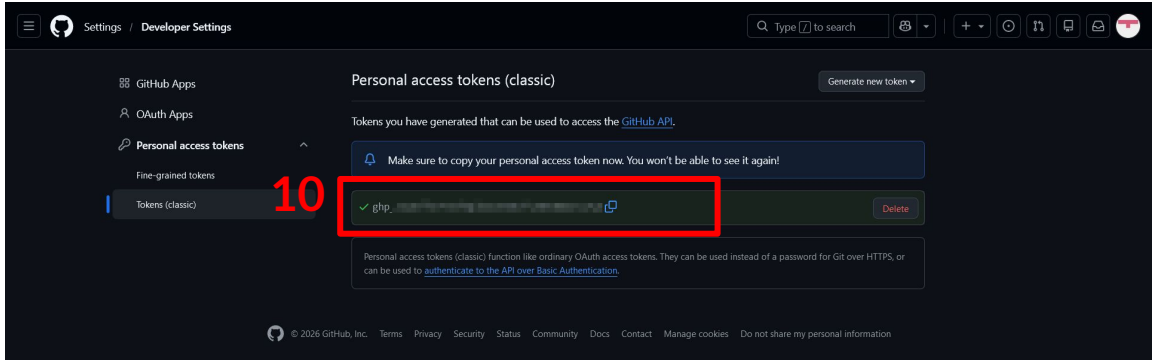
6. Anotar el propósito del token (opcional)
7. Definir la fecha de expiración (sin expiración es más práctico pero más inseguro)
8. Tildar el permiso **repo**
9. Deslizar abajo y pulsar **Generate Token**

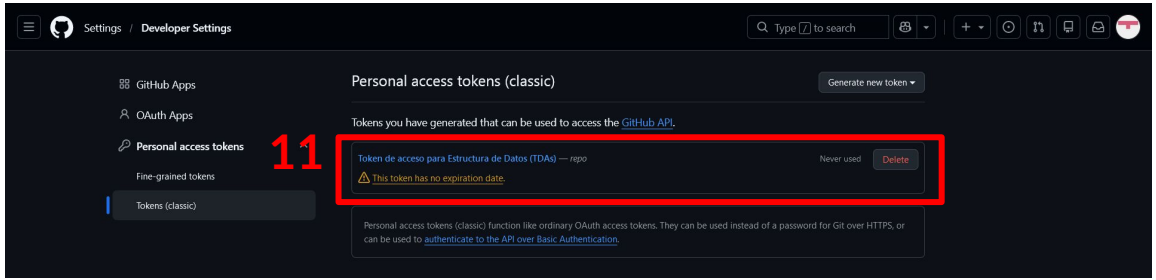


GitHub - Gestionando el acceso (tokens)

Para configurar un Personal Access Token (PAT) Clásico:

10. **Copiar y guardar el token generado** (al salir de esta pantalla no lo volveremos a ver).
11. Una vez finalizado el proceso tenemos el token activo. Si en el futuro hay que bloquear el acceso, el token puede revocarse con la opción **Delete** desde esta pantalla.





Clonando un repositorio remoto

Obteniendo los archivos - Clonar el repositorio

Una vez generado el token de acceso ya podemos replicar localmente el fork o el repositorio basado en template en nuestra computadora para poder trabajarlo. Este paso se logra mediante la clonación.

La operación de clonación crea una versión local de nuestro repositorio. Se logra mediante el comando **git clone**, ya sea aplicándolo de manera directa por línea de comandos, o de manera indirecta a través de las opciones del IDE seleccionado.

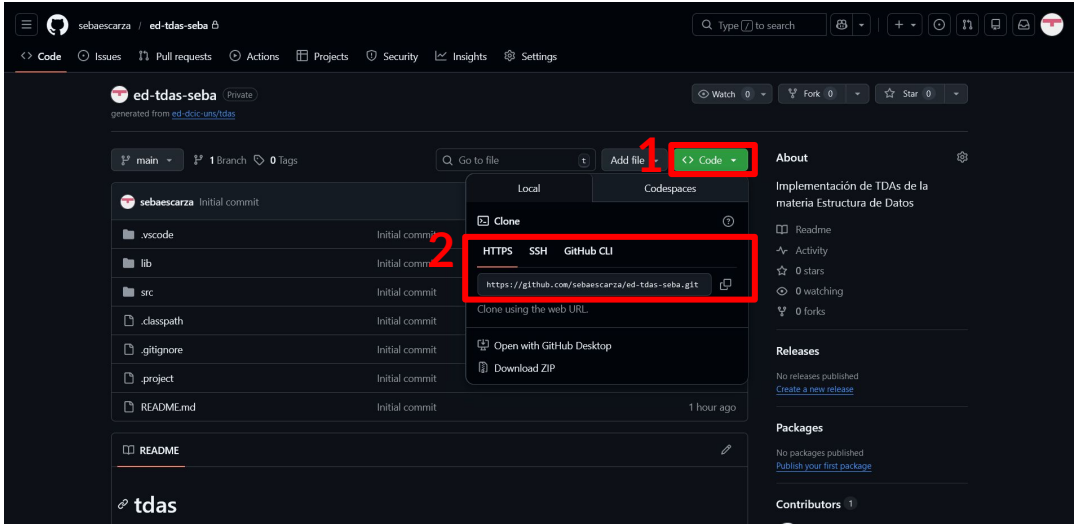
En lugar de simplemente descargar los archivos y carpetas como ocurre con la opción de descargar .zip, la clonación nos va a generar en nuestra PC un conjunto de carpetas y archivos configurados como un repositorio git.

Obteniendo los archivos - Clonar el repositorio

Para clonar un repositorio necesitamos tener instalado git en nuestra computadora y utilizar la dirección de clonación del repositorio.

Para instalar git ir al sitio https://git-scm.com descargarlo e instalarlo.

La dirección de clonación se copia del repositorio en GitHub, desde el menú **<> Code** en la sección HTTPS.

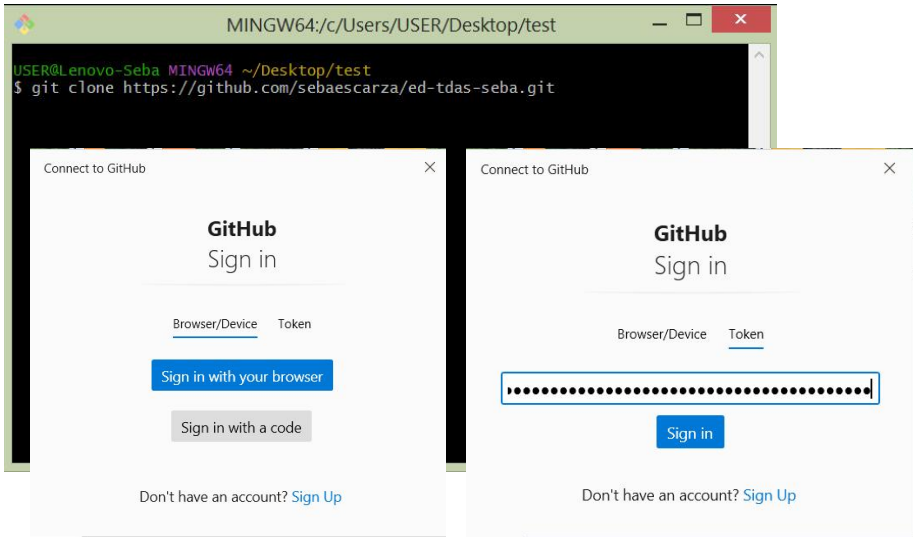


Clonando un repositorio remoto mediante CLI

Clonar el repositorio - Línea de comandos (CLI)

El uso de la línea de comandos (Command Line Interface - CLI) es ilustrativo de lo que los IDEs hacen por nosotros y normalmente no lo usaremos de esta manera.

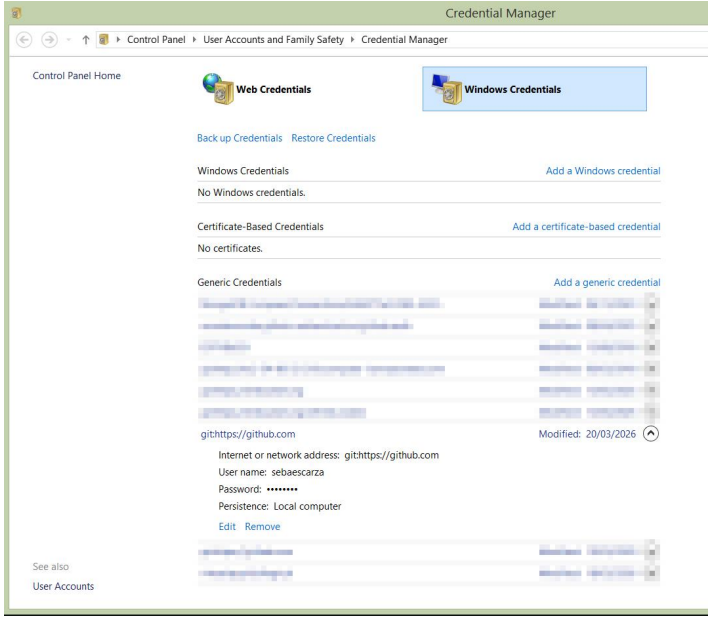
1. Abrir **Git Bash** y posicionarse en la carpeta que contendrá a la carpeta del repositorio. Ejecutar el comando **git clone** con la dirección de clonación del copiada del repositorio.
2. Si nunca nos hemos autenticado en esta PC se abrirá una ventana de autenticación de GitHub. Elegir la opción **Token**, pegar el token de acceso generado y pulsar **Sign in**.



Clonar el repositorio - Línea de comandos (CLI)

La ventana **Connect to GitHub** que aparece guarda las credenciales de acceso con el PAT en el **Credential Manager** de Windows, y no se nos vuelve a solicitar el PAT en ninguna operación subsiguiente.

Si en algún momento se necesita revocar el PAT, o se necesita eliminarlo de la PC para que se nos vuelva a solicitar y utilizar otro, se puede eliminar buscándolo en Windows Credentials, con la opción **Remove**.

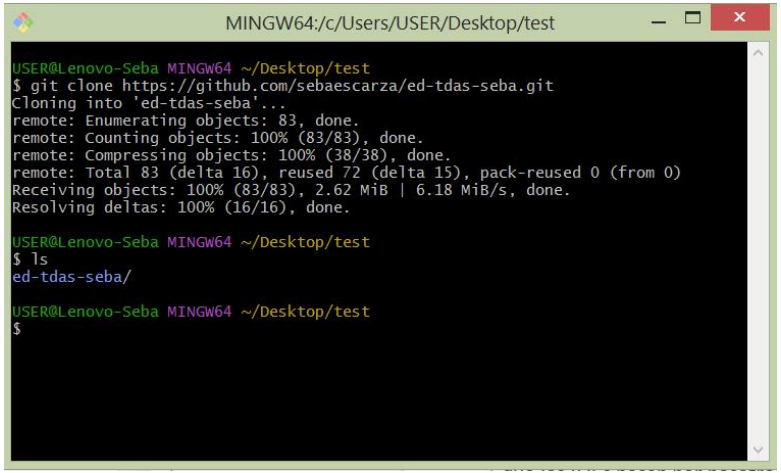


Clonar el repositorio - Línea de comandos (CLI)

El uso de la línea de comandos (Command Line Interface - CLI) es ilustrativo de lo que los IDEs hacen por nosotros y normalmente no lo usaremos de esta manera.

3. Si el proceso fue exitoso, git clonará el repositorio remoto en nuestra PC, en una subcarpeta de igual nombre que el repositorio, a partir de la carpeta donde estábamos posicionados.

Nota: Git Bash en Windows emula un subsistema Linux vía MINGW64 y utiliza comandos de dicho Sistema Operativo.



Clonando un repositorio remoto mediante Visual Studio Code

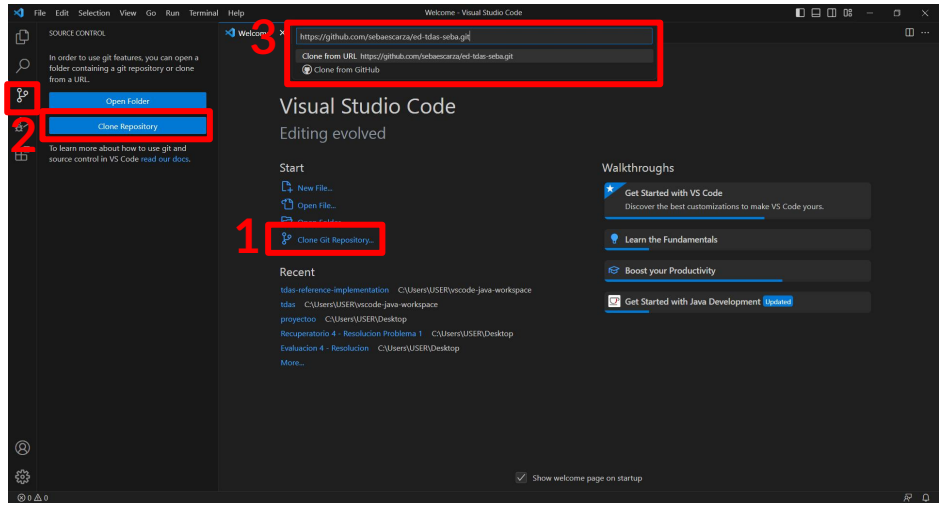
Clonar el repositorio - Visual Studio Code

Visual Studio Code nos permite crear un proyecto a partir de clonar un repositorio Git.

Esto se puede hacer desde la pantalla de bienvenida (1), o desde el menú de Git (2).

Pegar la dirección de clonación del repositorio y seleccionar **Clone from URL...**

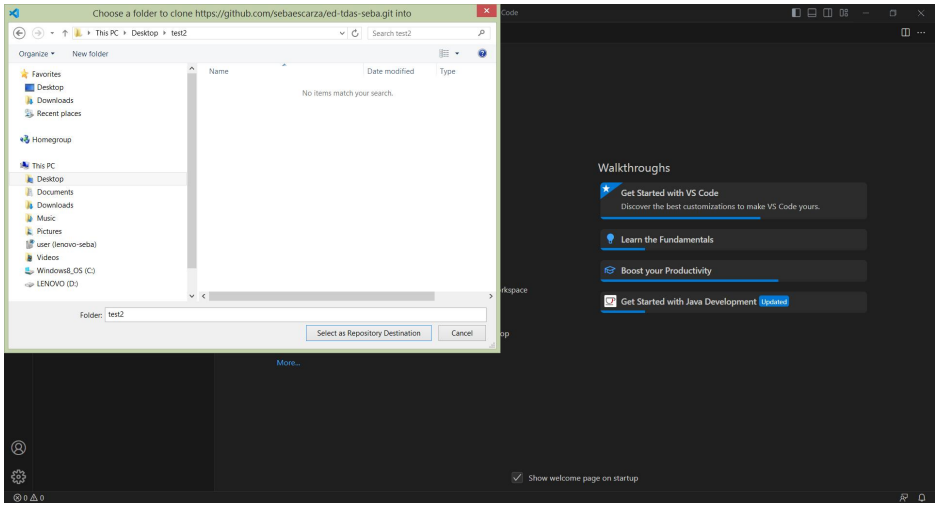
Si estas opciones no aparecen puede deberse a que no se instaló Git en la computadora local.



Clonar el repositorio - Visual Studio Code

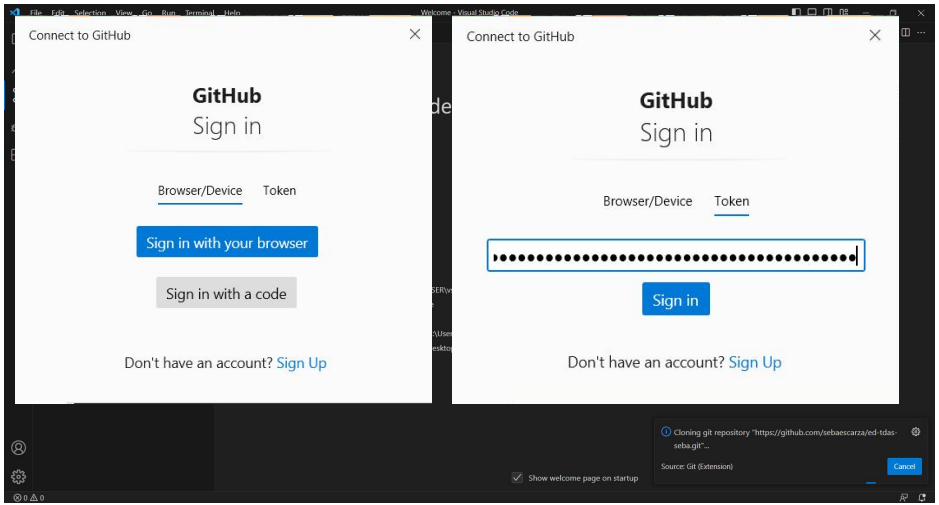
A continuación se nos pedirá que seleccionemos la carpeta en la cual se colocará la carpeta con el repositorio clonado.

Al hacerlo, se procederá a clonar el repositorio y si es privado y necesario, a pedirnos el token de acceso.



Clonar el repositorio - Visual Studio Code

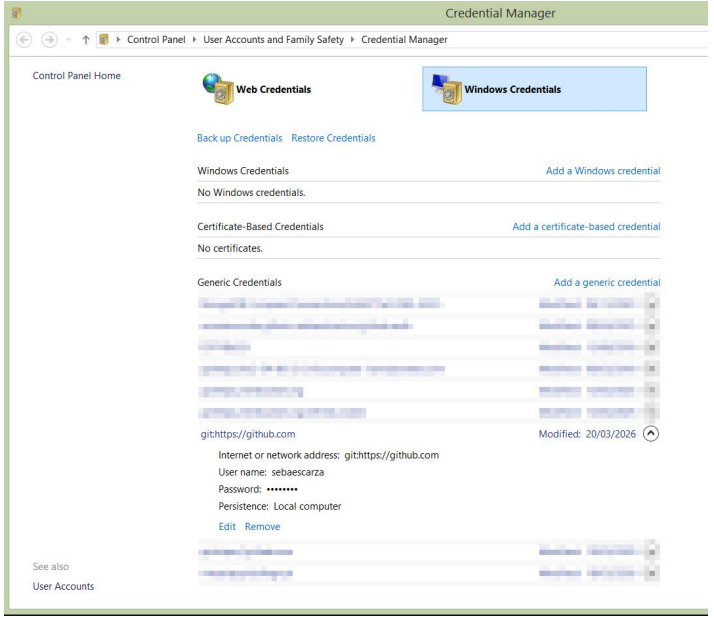
Si nunca nos hemos autenticado en esta PC se abrirá una ventana de autenticación de GitHub. Elegir la opción **Token**, pegar el token de acceso generado y pulsar **Sign in**.



Clonar el repositorio - Visual Studio Code

La ventana **Connect to GitHub** que aparece guarda las credenciales de acceso con el PAT en el **Credential Manager** de Windows, y no se nos vuelve a solicitar el PAT en ninguna operación subsiguiente.

Si en algún momento se necesita revocar el PAT, o se necesita eliminarlo de la PC para que se nos vuelva a solicitar y utilizar otro, se puede eliminar buscándolo en Windows Credentials, con la opción **Remove**.

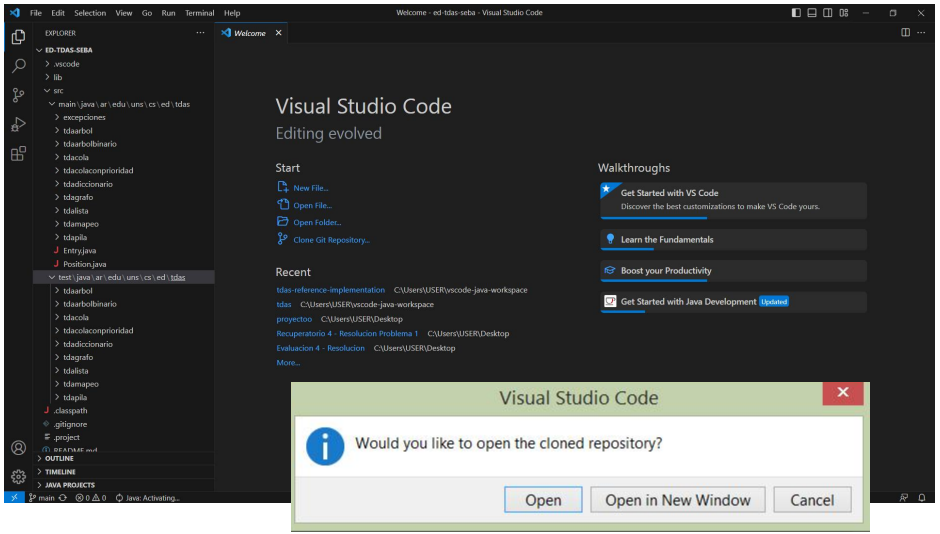


Clonar el repositorio - Visual Studio Code

Si el proceso es exitoso, se nos preguntará si queremos abrir el repositorio clonado.

Al abrirlo tendremos el repositorio en Visual Studio Code configurado como un proyecto listo para utilizar.

Siempre que se abra la carpeta del repositorio con la opción **Open Folder**, tendremos acceso al proyecto como repositorio git sin necesidad de volver a clonar nada.



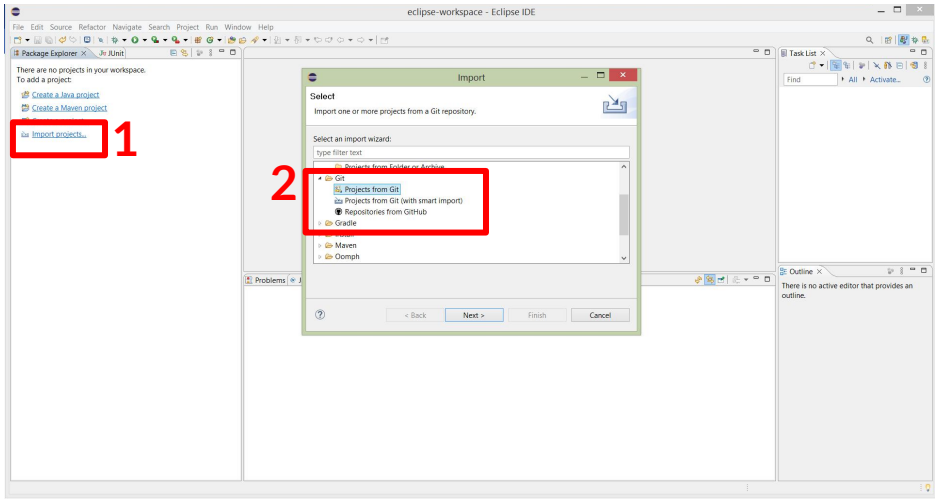
Clonando un repositorio remoto mediante Eclipse

Clonar el repositorio - Eclipse

Eclipse nos permite crear un proyecto a partir de clonar un repositorio Git.

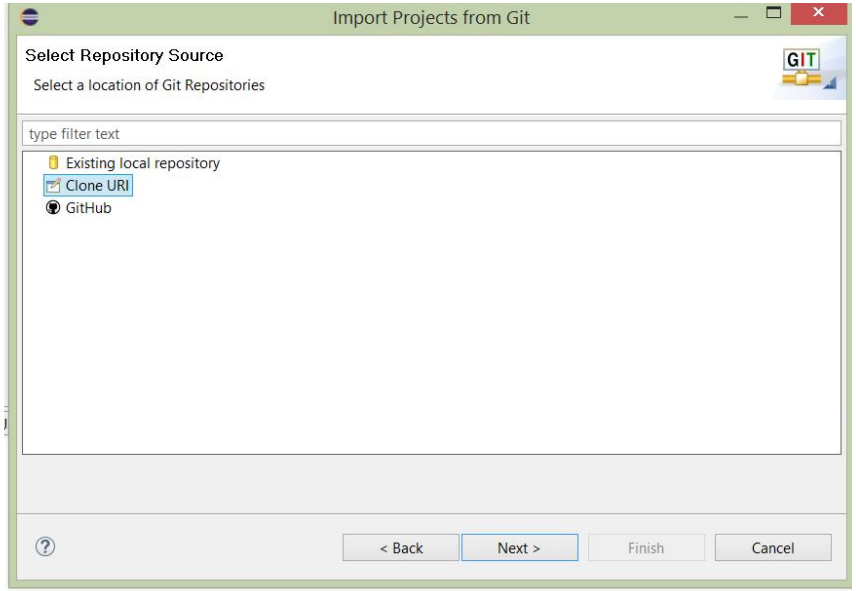
Esto se puede hacer desde **Import projects...** en el Package Explorer (1), o desde el menú **File > Import...**.

Luego hay que scrollear y elegir **Git > Projects from Git** (2) y **Next >**.



Clonar el repositorio - Eclipse

Elegir **Clone URI**, y luego **Next >**



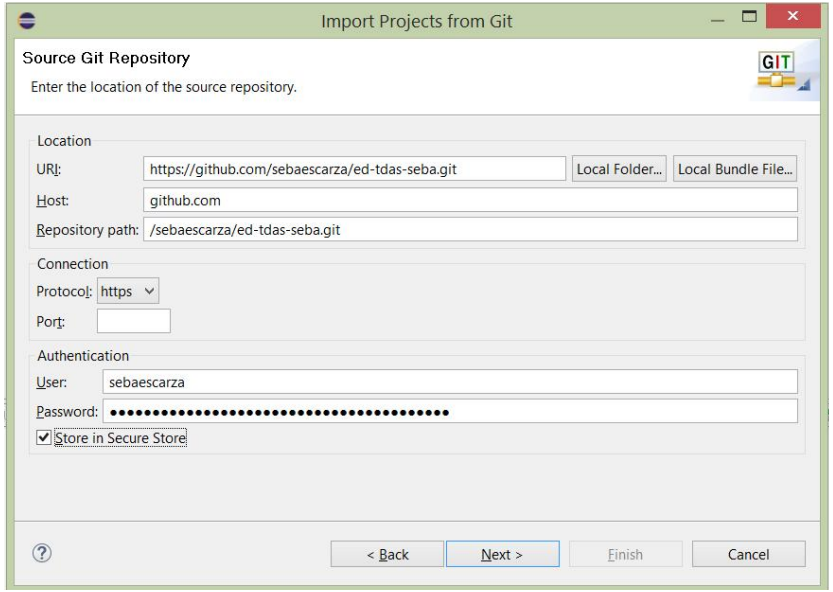
Clonar el repositorio - Eclipse

En esta pantalla se pega la dirección de clonación del repositorio en **URI**.

En la sección de **Authentication**, se completa el **User** (nuestro usuario de GitHub, si no aparece pre-cargado) y en **Password** se pega el token de acceso.

Tildando **Store in Secure Store** Eclipse guarda la credencial y nos evita tener que cargarla cada vez.

Eclipse no usa el Credential Manager de Windows para esto, sino un store propio.

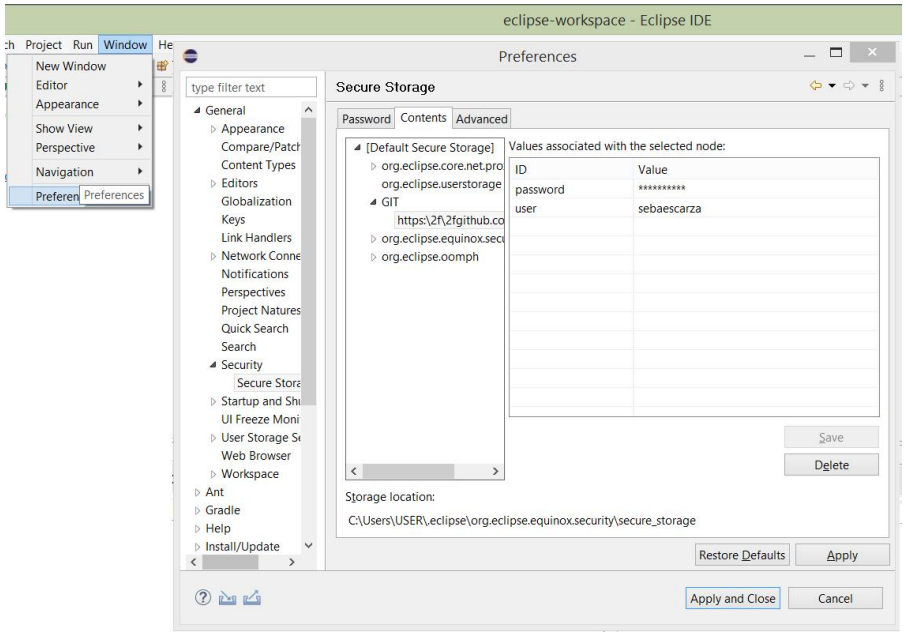


Clonar el repositorio - Eclipse

Si en algún momento se necesita revocar el PAT, o se necesita eliminarlo del store de Eclipse para que se nos vuelva a solicitar y utilizar otro token, se puede eliminar siguiendo la siguiente secuencia de opciones y comandos:

Menú **Window > Preferences > General > Security > Secure Storage > Contents > [Default Secure Storage]** > **GIT** > Seleccionar la URL (confirmar con el campo user) > **Delete**.

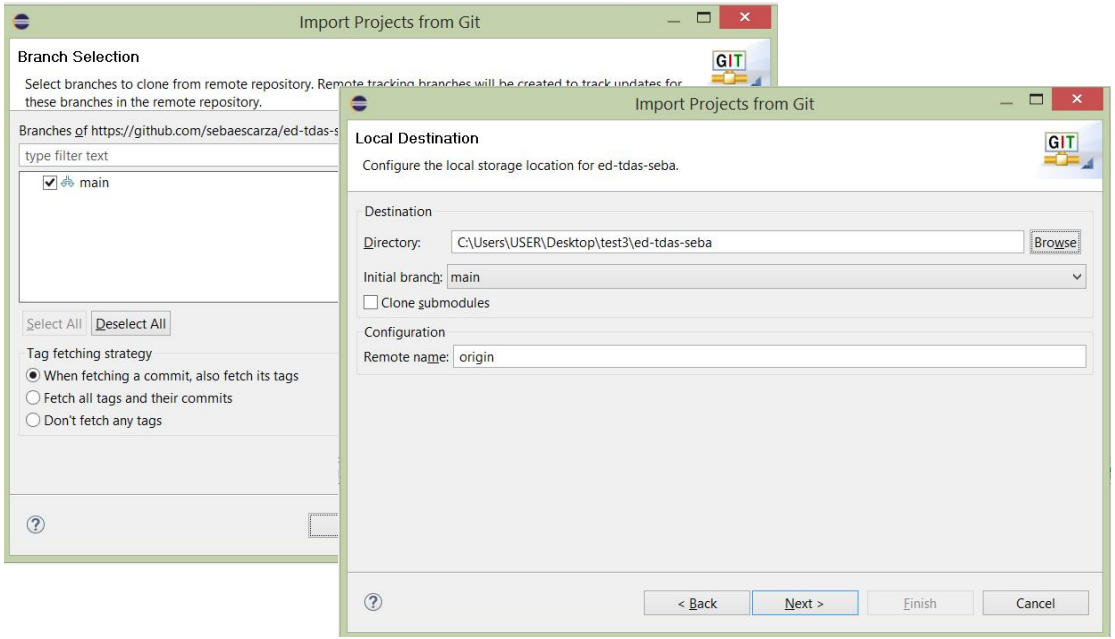
Luego **Apply > Apply and Close** y no olvidar de **reiniciar Eclipse**.



Clonar el repositorio - Eclipse

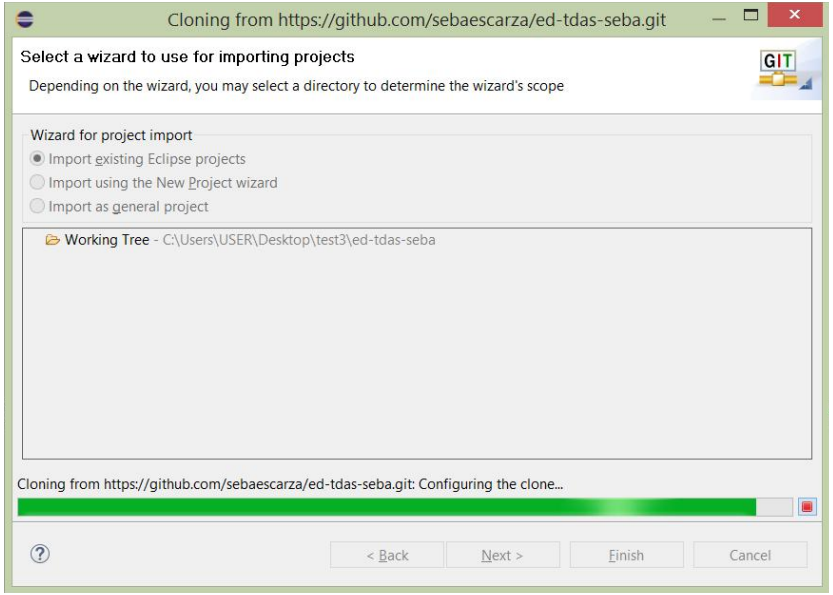
En la siguiente pantalla dejamos las opciones por defecto para clonar la rama **main** y usar la estrategia de tag fetching estándar.

Al pulsar **Next>**. debemos seleccionar la carpeta (**Directory**) dentro de la cual Git creará la carpeta con el repositorio clonado. Dejamos las demás opciones como están y pulsamos **Next>**.



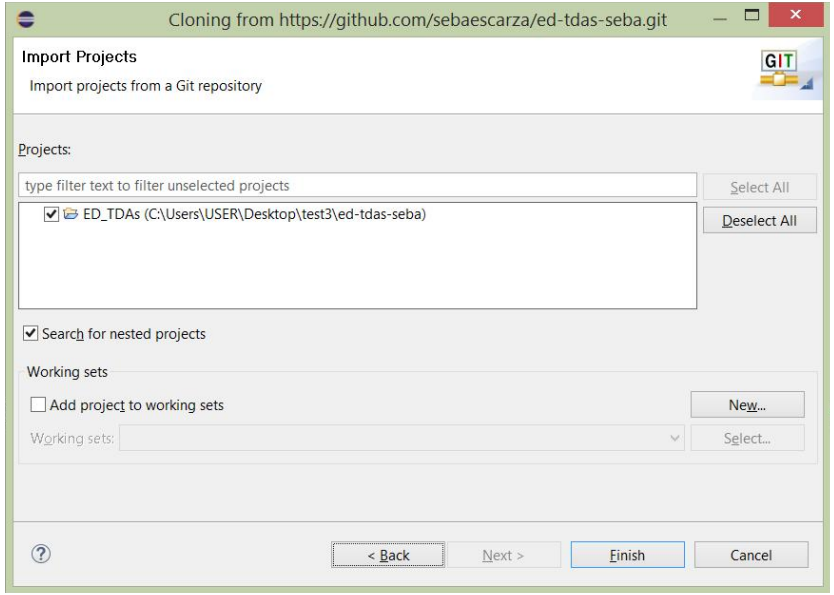
Clonar el repositorio - Eclipse

A continuación Git procede a clonar localmente el repositorio en la carpeta elegida.



Clonar el repositorio - Eclipse

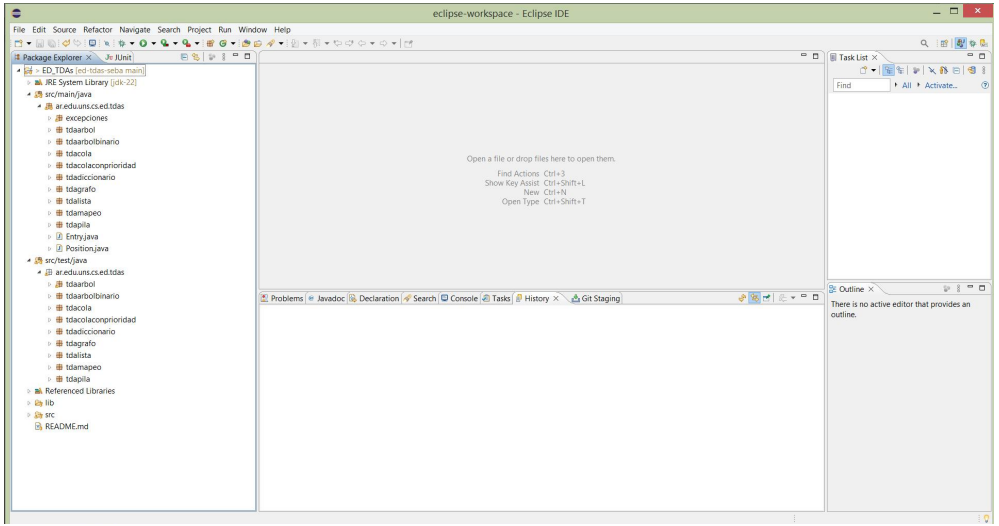
Como paso final, se nos pregunta si queremos importar el proyecto de Eclipse clonado al IDE. Dejamos las opciones con el proyecto seleccionado para hacerlo, y pulsamos **Finish** para terminar.



Clonar el repositorio - Eclipse

Si todo salió bien, tendremos el repositorio en Eclipse configurado como un proyecto listo para utilizar.

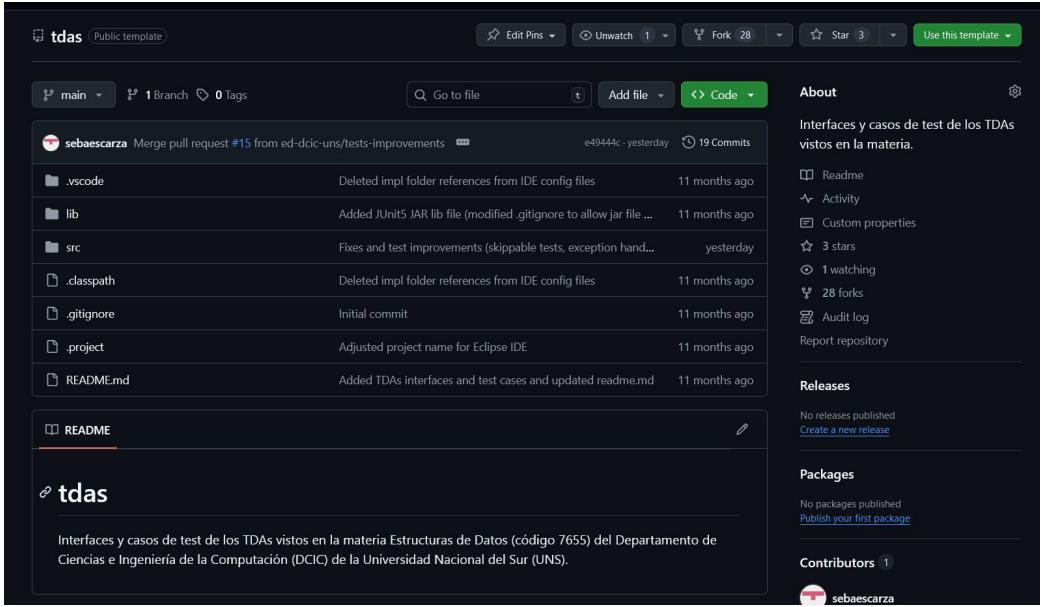
Siempre que se abra el proyecto en la carpeta del repositorio con Eclipse, tendremos acceso al mismo como repositorio git sin necesidad de volver a clonar nada.



Estructura y organización del repositorio de TDAs

Estructura del repositorio

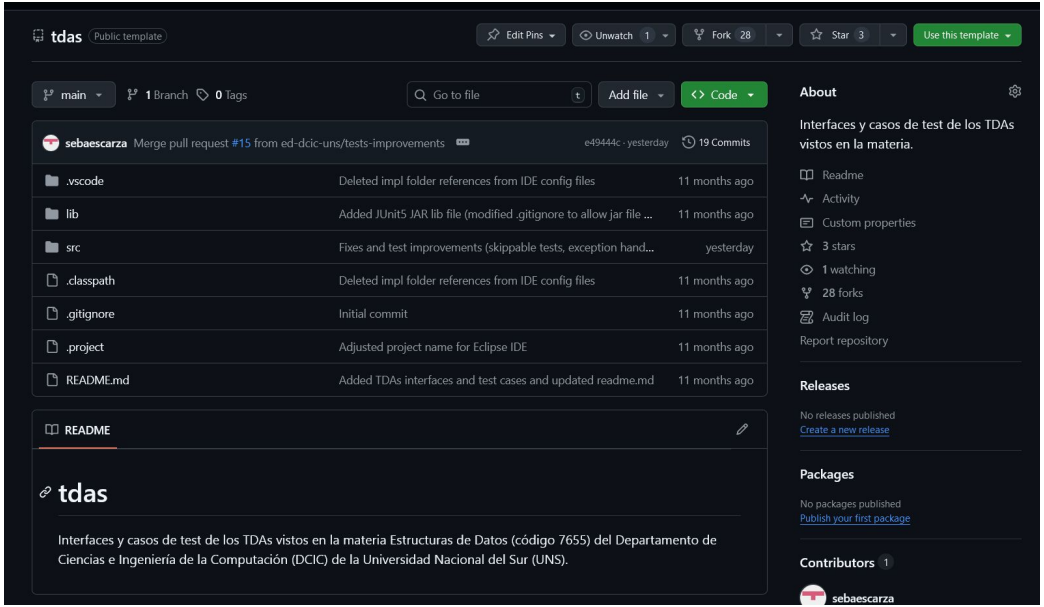
Una vez obtenidos los archivos, ya sea descomprimiendo el archivo .zip, o clonando localmente los repositorios creados remotamente en GitHub (fork o nuevo en base a template), la estructura de carpetas y archivos obtenida es la misma del repositorio de TDAs público original.



Estructura del repositorio

Carpetas:

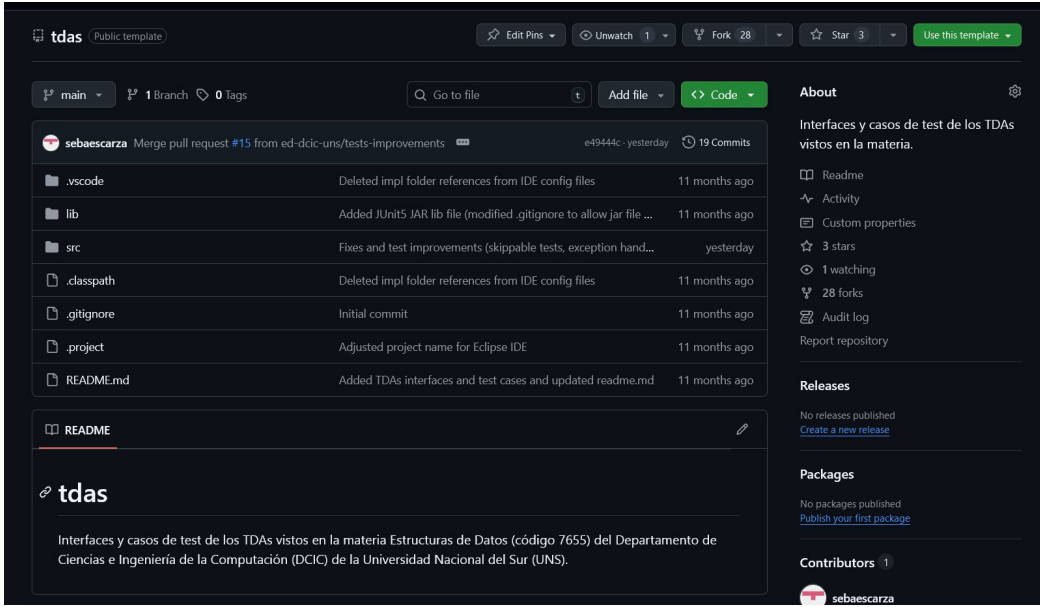
- **.vscode**: archivos de configuración para que el repositorio funcione como un proyecto de VSCode
- **lib**: contiene el .jar de JUnit
- **src**: contiene los archivos fuente con las interfaces y tests de los TDAs



Estructura del repositorio

Archivos en carpeta raíz:

- **.classpath**: configuración de ubicaciones para Eclipse. VSCode también lo puede utilizar
- **.gitignore**: configuración de git de archivos a ignorar
- **project**: Archivo para que el repositorio funcione como un proyecto de Eclipse
- **README.md**: Un archivo Markdown con información del proyecto (se muestra debajo en GitHub)

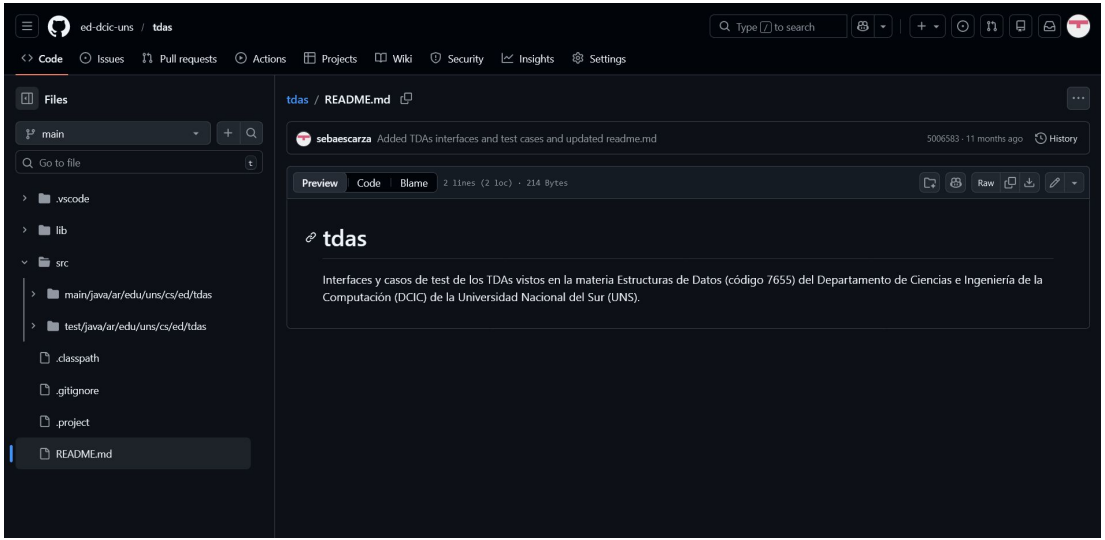


Estructura del repositorio

Código fuente en **src**. Contiene dos rutas principales en las cuales se encuentran las carpetas correspondientes a los paquetes del repositorio:

- **main/java**: interfaces de TDAs
- **test/java**: tests de TDAs

Los IDEs usan como base estas carpetas para encontrar los archivos **.java** en los paquetes.



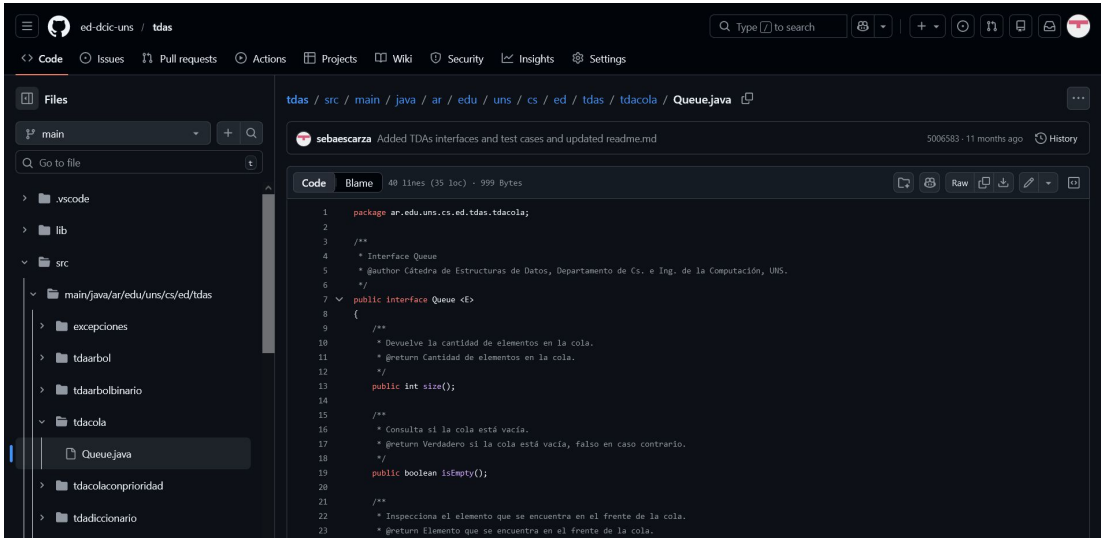
Estructura del repositorio

Código fuente en **src**.

Ej. interface **Queue<E>**

En el paquete:
ar.edu.uns.cs.ed.tdas.tdacola,

Ruta:
src/main/java/**ar/edu/uns/cs/ed/tdas/tdacola/Queue.java**



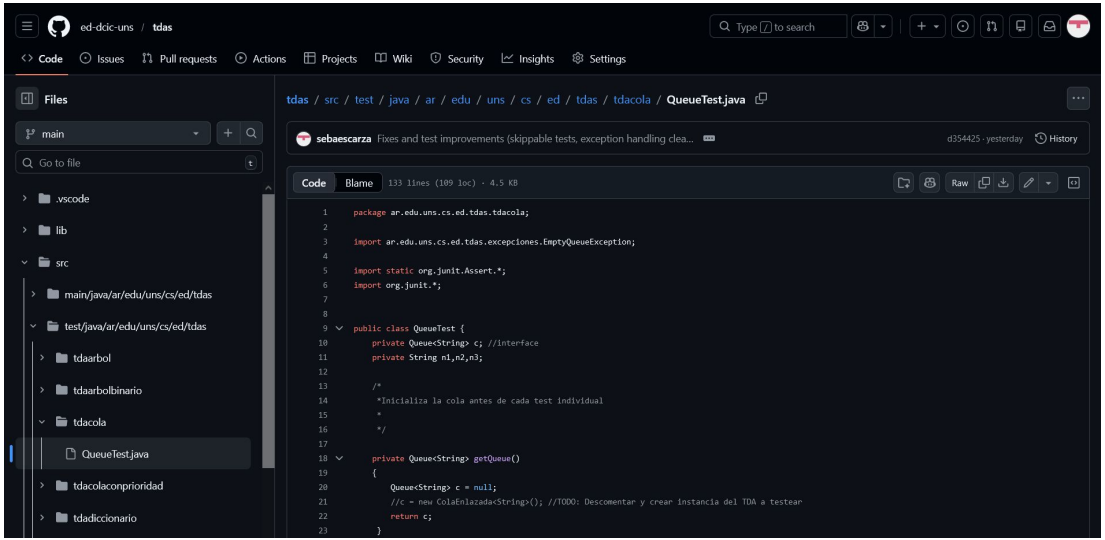
Estructura del repositorio

Código fuente en **src**.

Ej. clase tester **QueueTest**

En el paquete:
ar.edu.uns.cs.ed.tdas.tdacola,

Ruta:
src/test/java/**ar/edu/uns/cs/ed/tdas/tdacola/QueueTest.java**



Estructura del repositorio

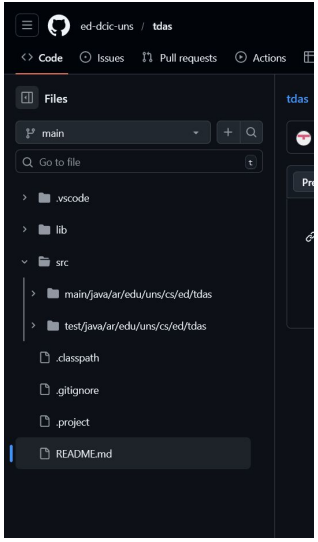
Código fuente en **src**:

- **src/main/java**: interfaces de TDAs
- **src/test/java**: tests de TDAs

La separación en **main** vs **test** permite separar el código de la aplicación (TDAs) del código de testeo y eventualmente desplegar sólo la aplicación sin los tests.

La configuración de proyecto considera ambas ubicaciones como equivalentes para definir las rutas de paquetes, lo que provoca que cada TDA y su tester pertenezcan al mismo paquete y sean visibles sin necesidad de hacer el **import**.

Nota: No se debe confundir esta carpeta **main**, con la rama principal **main** de git.



Recomendaciones finales

Algunas recomendaciones finales

A lo largo de esta presentación se vieron varias alternativas para obtener los archivos del Repositorio de TDAs de la materia: descarga .zip, clonar un fork, y **clonar un repositorio basado en template,** siendo esta última la alternativa recomendada.

Como parte del proceso se mostró cómo crear/configurar una cuenta en Git, definiendo tokens de acceso, y cómo clonar repositorios remotos mediante CLI, Visual Studio Code y Eclipse.

El uso de repositorios Git locales clonados desde GitHub permite gestionar mejor los cambios de código y contar con un backup online del código desarrollado. Próximamente veremos una introducción a Git y GitHub para utilizar estas herramientas de manera efectiva.

Algunas recomendaciones finales

En relación al trabajo en la materia y a la organización de archivos, una vez obtenidos los archivos en la PC de manera local, se cuenta con una estructura de carpetas y paquetes que conviene aprovechar y utilizar de manera consistente, para lo cual se recomienda que:

- Coloquen las implementaciones de cada TDA en la carpeta/paquete de la interface que implementan: **src/main/java/ar/edu/uns/cs/ed/tdas/.../**
- Desarrollar los ejercicios que involucran agregar métodos a los TDAs dentro de los archivos que implementan dichos TDAs.
- Incluir los ejercicios que usan los TDAs en paquetes/carpetas incluidos en el proyecto a partir de la ruta **src/main/java/**. Un buen ejemplo es disponer los archivos **.java** en carpetas/paquetes por práctico y ejercicio. Ej. **src/main/java/tp3/ejercicio5/**.

Si respetan la correspondencia carpetas/paquetes lograrán un sistema con todos los archivos accesibles mediante **import**s. Es importante usar minúsculas y evitar signos especiales para nombrar paquetes/carpetas.

¡A programar y testear!